

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 24.М41-мм

Метод преобразования квадратичного внимания в линейное для больших языковых моделей

Щукин Илья Вячеславович

Отчёт по преддипломной практике
в форме «Решение»

Научный руководитель:
к. ф.-м. н., ассистент кафедры информационно-аналитических систем Г. А. Чернышев

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	5
2. Обзор	6
2.1. Механизм внимания	6
2.2. Существующие решения	8
3. Реализация	15
3.1. Описание метода	15
3.2. Инструмент для выбора слоёв	17
3.3. Модификация инфраструктуры	20
3.4. Результат	23
4. Эксперименты	24
4.1. Исследовательские вопросы	24
4.2. Дизайн экспериментов	24
4.3. Описание используемых бенчмарков	25
4.4. Обучение модели	26
4.5. Результаты обучения	31
4.6. Оценка производительности	32
4.7. Выводы	33
Заключение	36
Список литературы	37

Введение

Механизм внимания, описанный в статье «Attention Is All You Need» [1] и широко применяемый в современных больших языковых моделях обладает существенными недостатками: квадратичной сложностью вычислений по длине последовательности $O(L^2)$ и линейно растущим потреблением памяти для KV-Cache $O(L)$, используемого на этапе предсказания для сохранения промежуточных состояний внимания. Это создаёт серьёзные ограничения при работе с длинными последовательностями (свыше 256 тысяч токенов) как в процессе обучения, так и на этапе предсказания, что особенно критично для задач, требующих обработки больших контекстов, например для написания кода или для глубоких рассуждений.

Возможным решением этой проблемы может являться применение линейного внимания, позволяющее снизить вычислительную сложность до линейной по длине последовательности, а также сократить потребляемую память до константной. Однако линейное внимание в свою очередь также имеет важное ограничение в эффективности работы. Конечная память не позволяет модели чётко воспроизводить токены из контекста, из-за чего модель, использующая только линейное внимание, хуже справляется с задачами точного извлечения информации из длинного контекста. Поэтому в современных моделях, таких как Qwen3.5 [26] и Kimi Linear [11], используется гибридная архитектура, сочетающая разные виды внимания в одной модели, например с помощью чередования слоёв. Такой подход всё ещё позволяет в несколько раз сократить общее потребление памяти и ускорить этап инференса, особенно на длинных последовательностях, при этом сохраняя исходные метрики и не снижая качества предсказаний.

Обучение больших языковых моделей с нуля с использованием новых архитектур внимания требует значительных вычислительных ресурсов и времени. Поэтому было принято решение о разработке метода, который бы преобразовывал уже обученные модели с квадратичным вниманием в модели с линейным вниманием. Также требовалось, чтобы необходимые

ресурсы на преобразование составляли малую часть от расходов на обучение с нуля.

В данной работе описан процесс реализации метода для преобразования моделей с квадратичным вниманием в гибридные модели с линейным, а также проведенные эксперименты. Полученная модель-студент демонстрирует качество генерации, сопоставимое с исходной моделью-учителем при уменьшении потребления памяти в 5 раз и значительном ускорении в процессе предсказаний на 19,82% в типовых сценариях использования и на 37,32% при работе с длинными последовательностями. На обучение модели-студента потребовалось 0,46% от вычислений, потраченных на предобученную модель, что говорит об эффективности предложенного метода.

1. Постановка задачи

Целью данной ВКР является разработка и экспериментальная оценка эффективности метода преобразования предобученной большой языковой модели в модель с линейным вниманием. Для достижения данной цели были сформулированы следующие задачи.

1. Провести обзор существующих решений для преобразования квадратичного внимания в линейное в предобученных моделях.
2. Разработать метод для преобразования внимания. Реализовать все необходимые инфраструктурные компоненты.
3. Преобразовать модель с помощью полученного метода и провести экспериментальное исследование. Оценить качество модели, а также скорость обучения и инференса.

2. Обзор

2.1. Механизм внимания

2.1.1. Квадратичное внимание

Механизм многоголового внимания, впервые описанный в работе «Attention Is All You Need» [1], является фундаментальным строительным блоком трансформерной архитектуры. В его основе лежит операция масштабированного скалярного произведения, которая позволяет модели динамически определять релевантность различных элементов входной последовательности друг относительно друга.

Для одиночной «головы» внимание вычисляется следующим образом:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V, \quad (1)$$

где Q (Query), K (Key) и V (Value) — матрицы запросов, ключей и значений соответственно, а d_k — размерность пространства ключей, используемая в качестве масштабирующего фактора.

Вместо выполнения единичной функции внимания над векторами размерности d_{model} , авторы предложили линейно проецировать запросы, ключи и значения h раз с использованием различных матриц весов. Это позволяет механизму параллельно извлекать информацию из различных подпространств представлений в разных позициях:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad (2)$$

где каждая отдельная голова вычисляется как:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \quad (3)$$

Где $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ — матрицы проекций, $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ — итоговая матрица агрегации.

Несмотря на высокую выразительную способность и возможность эффективного распараллеливания на современных графических ускорителях, стандартный механизм многоголового внимания опирается на

явное вычисление матрицы размерности $L \times L$, где L — длина контекста. Как следствие, алгоритм имеет критическое алгоритмическое ограничение: вычислительную сложность $\mathcal{O}(L^2)$ и линейно растущее потребление памяти для хранения KV-кэша $\mathcal{O}(L)$, что делает его применение для обработки сверхдлинных последовательностей крайне ресурсоёмким и требует перехода к методам линеаризации внимания.

2.1.2. Линейное внимание

Классическое линейное внимание [28] может быть представлено следующей рекуррентной формулой:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top \in \mathbb{R}^{d \times d}, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \in \mathbb{R}^d,$$

где $\mathbf{S}_t \in \mathbb{R}^{d \times d}$ — это матрица состояния, которая обновляется по мере обработки контекста для каждого нового токена t . $\mathbf{v}_t$, $\mathbf{k}_t$ и $\mathbf{q}_t$ — это векторы значений, ключей, и запросов токена t . $\mathbf{o}_t$ — это результат вычисления внимания для токена t . d — это размер одной головы внимания.

Линейное внимание без модификаций значительно уступает обычному. Чтобы справиться с этим недостатком была предложена архитектура Mamba 2 [5], в которой к линейному вниманию добавлен механизм затухания, позволяющий модели забывать предыдущую информацию.

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top,$$

где $\alpha_t \in (0, 1)$ — коэффициент затухания, зависящий от входных данных.

Недостатком данного подхода является невозможность точечного удаления информации, поскольку сохранённая история затухает равномерно.

Альтернативный подход используется в DeltaNet [23], где применяется дельта-правило [29]. Этот механизм позволяет выборочно обновлять память, заменяя старое значение на новое для конкретного ключа:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top,$$

где β_t — коэффициент, определяющий силу перезаписи.

Это обеспечивает высокую точность извлечения данных, но модели не хватает способности быстро очистить всю память целиком, например, при резкой смене контекста.

В работе [31] описана архитектура для линейного внимания — Gated DeltaNet, которая объединяет в себе преимущества механизма затухания и дельта-правила.

$$\mathbf{S}_t = \mathbf{S}_{t-1} (\alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top.$$

Для данной ВКР в качестве линейного внимания был выбран Gated DeltaNet, поскольку он показывает наилучшие результаты на различных задачах, особенно на тех, где необходимо точно извлекать информацию из контекста. В таблице 1 приведены метрики для разных архитектур внимания на бенчмарке NIAH [24], который оценивает точность извлечения данных из длинных последовательностей. В таблице задачи из NIAH упорядочены по увеличению сложности. Видно, что с увеличением сложности другие методы справляются хуже на данном бенчмарке.

Таблица 1: Сравнение метрик на NIAH для разных архитектур линейного внимания. Источник [31].

Model	S-NIAH-1 (pass-key retrieval)				S-NIAH-2 (number in haystack)				S-NIAH-3 (uuid in haystack)		
	1K	2K	4K	8K	1K	2K	4K	8K	1K	2K	4K
DeltaNet	97.4	96.8	99.0	98.8	98.4	45.6	18.6	14.4	85.2	47.0	22.4
Mamba2	99.2	98.8	65.4	30.4	99.4	98.8	56.2	17.0	64.4	47.6	4.6
Gated DeltaNet	98.4	88.4	91.4	91.8	100.0	99.8	92.2	29.6	86.6	84.2	27.6

2.2. Существующие решения

2.2.1. LoLCATs

В работе [12] предложен двухэтапный метод LoLCATs (Low-rank Linear Conversion with Attention Transfer) для замены квадратичного внимания на линейный аналог в предобученных моделях. В качестве линейного внимания авторы используют Kernel Attention [28], ключевой идеей которого является аппроксимация функции softmax с помощью

скалярное произведения отображений. Рассмотрим формулу вычисления квадратичного внимания:

$$a_{n,i} = \frac{\exp\left(q_n^\top k_i / \sqrt{d}\right)}{\sum_{i=1}^n \exp\left(q_n^\top k_i / \sqrt{d}\right)}, \quad y_n = \sum_{i=1}^n a_{n,i} v_i, \quad \text{for } n \in [1, \dots, l].$$

Авторы отмечают, что экспоненты в формуле внимания являются ядерными функциями и могут быть приближены с помощью скалярного произведения двух отображений.

$$\hat{y}_n = \sum_{i=1}^n \frac{\varphi(\mathbf{q}_n)^\top \varphi(\mathbf{k}_i) \mathbf{v}_i}{\sum_{i=1}^n \varphi(\mathbf{q}_n)^\top \varphi(\mathbf{k}_i)} = \frac{\varphi(\mathbf{q}_n)^\top \left(\sum_{i=1}^n \varphi(\mathbf{k}_i) \mathbf{v}_i^\top\right)}{\varphi(\mathbf{q}_n)^\top \sum_{i=1}^n \varphi(\mathbf{k}_i)}.$$

С помощью такой замены они получают линейное внимание и рекуррентное представление.

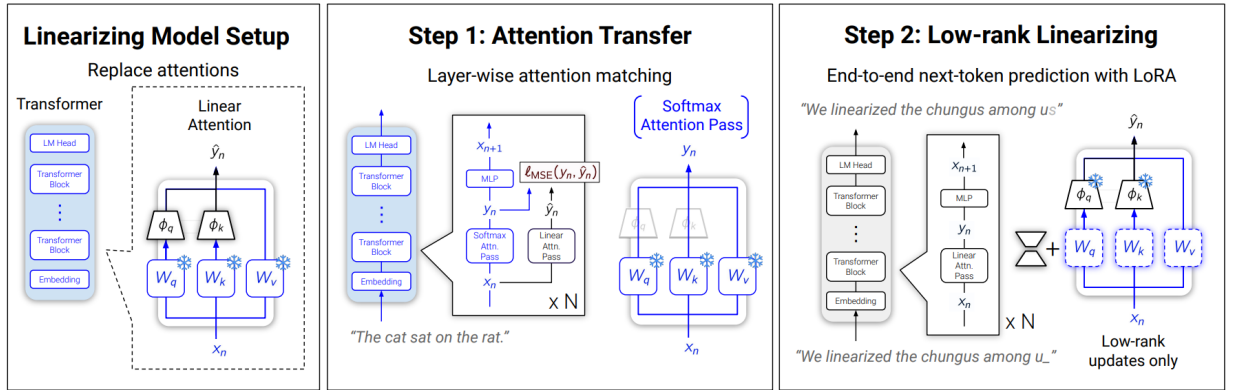


Рис. 1: Метод LoLCATs. Источник [12].

Предложенный метод состоит из двух шагов, на рис. 1 представлена схема метода. На первом этапе линейное внимание обучается приближать выходы квадратичного внимания. В качестве выхода слоя внимания дальше используется выход исходного внимания, а не линейного, что позволяет не накапливать ошибку при прямом проходе (teacher forcing). В качестве функции потерь используется Mean Squared Error (MSE) над

выходами исходного внимания и линейного.

$$\ell_{\text{MSE}} = \frac{1}{MH} \sum_{m=1}^M \sum_{h=1}^H \ell_{\text{MSE}}^{h,m} \quad , \quad \ell_{\text{MSE}}^{h,m} = \frac{1}{d} \sum_{n=1}^d (\mathbf{y}_n - \hat{\mathbf{y}}_n)^2,$$

где M — число слоёв модели, H — число голов внимания, y_n — выход исходного внимания и $\hat{y}$ — выход линейного внимания. В процессе обучения исходные параметры модели заморожены, а обучаются только небольшие отображения φ .

На втором этапе происходит дообучение модели. Для этого авторы используют LoRA [14] адаптеры для матриц W_Q, W_K, W_V, W_O . Метод LoRA добавляет к исходным параметрам дополнительные матрицы, использующие низкоранговое представление. При обучении обучаются только добавленные параметры, а исходные остаются замороженными, это позволяет сэкономить вычислительные ресурсы во время дообучения.

Проведённые авторами эксперименты показали значительное падение метрик относительно исходной модели в данной конфигурации. Для решения этой проблемы было предложено использовать комбинацию линейного внимания с Sliding Window Attention, то есть с квадратичным вниманием ограниченным небольшим окном длиной 64 токена. Оно в свою очередь также использует константную память и требует линейное время для вычислений.

$$\hat{y}_n = \frac{\sum_{i=n-w+1}^n \gamma \exp\left(\frac{q_n^\top k_i}{\sqrt{d}} - c_n\right) v_i + \varphi_q(q_n)^\top \left(\sum_{j=1}^{n-w} \varphi_k(k_j) v_j^\top\right)}{\sum_{i=n-w+1}^n \gamma \exp\left(\frac{q_n^\top k_i}{\sqrt{d}} - c_n\right) + \varphi_q(q_n)^\top \left(\sum_{j=1}^{n-w} \varphi_k(k_j)\right)}.$$

Метод LoLCATs позволяет сохранить исходные значения метрик на представленных авторами бенчмарках кроме MMLU [16]. Для этого бенчмарка происходит падение на 18% для Mistral 7B [17] и 21% для Llama3 8B [33], что является значительной разницей. Также авторы приводят результаты для задачи Passkey [19]. С этой задачей линейризованная модель не справляется и показывает нулевые значения в

отличие от исходной модели, значения для которой близки к 100. Это демонстрирует существенный недостаток линейного внимания, а именно константная память, которая не способна вместить в себя весь контекст и точно извлекать из него необходимые токены. Модификации линейного внимания, например Gated DeltaNet, позволяют улучшить данную способность.

2.2.2. Jet-Nemotron

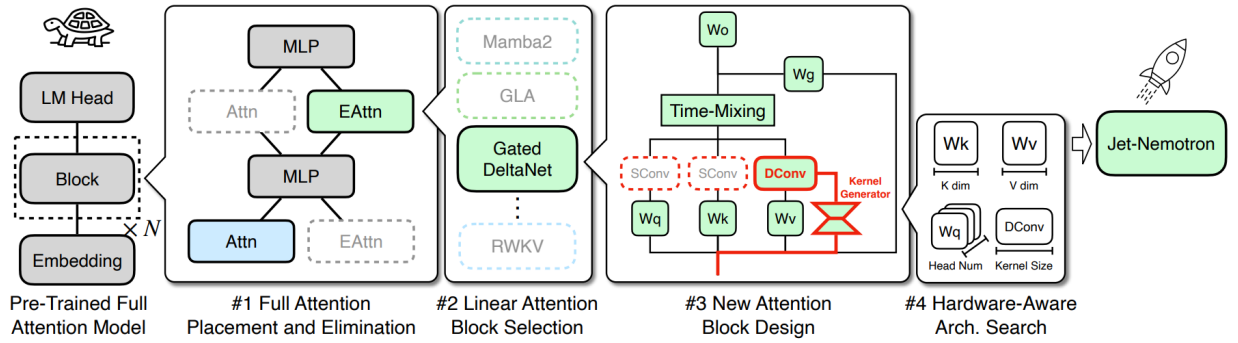


Рис. 2: Метод Jet-Nemotron. Источник [10].

В данной работе [10] описан метод PostNAS для получения моделей с линейным вниманием из предобученных моделей. В предложенном подходе используется гибридная архитектура моделей, в которой слои с квадратичным вниманием чередуются с линейным, таким образом становится возможным сохранить исходное качество и всё ещё значительно ускорить этап предсказаний. Метод PostNAS, изображённый на рис. 2, состоит из 4 шагов:

1. На первом этапе происходит обучение модели с помощью послылой дистиляции. В процессе дистиляции происходит случайное переключение между исходным вниманием и линейным GLA [8]. Таким образом получается универсальная модель, способная работать в разных конфигурациях. Она используется для того, чтобы выбирать важные слои. С помощью лучевого поиска авторы находят слои важные для трёх задач: MMLU [16], Retrieval и Math. После этого они оставляют только наиболее важные слои для этих

задач. На рис. 3 представлена карта важности слоёв в зависимости от задачи, а также сравнение качества модели на MMLU в зависимости от числа слоёв с квадратичным вниманием.

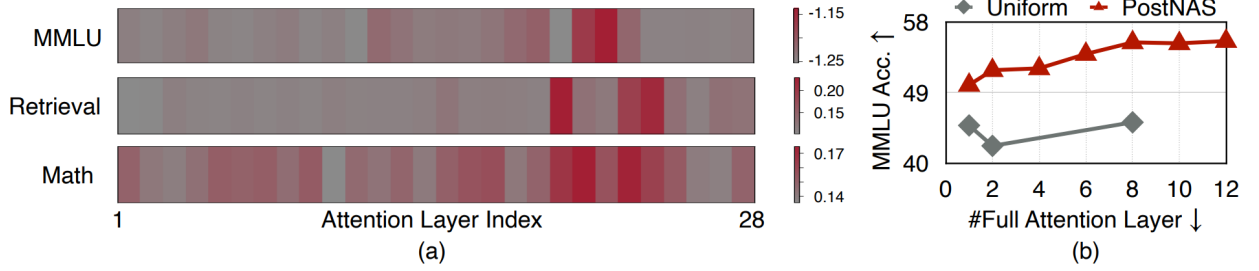


Рис. 3: Слева: карты важности слоёв для разных задач. Справа: точность ответов на бенчмарке MMLU [16] в зависимости от числа слоёв с квадратичным вниманием и в зависимости подхода к выбору слоёв. Источник [10].

2. На втором этапе происходит выбор конкретного варианта линейного внимания. По результатам сравнительного анализа наилучшие показатели продемонстрировала архитектура Gated DeltaNet [31] и была выбрана для дальнейших экспериментов.
3. После выбора Gated DeltaNet авторы предлагают модифицировать его за счёт добавления свёрток с параметрами, обусловленными входными данными.
4. Для определения гиперпараметров, такие как размеры голов и их количество для линейного внимания проводится поиск, где целевой метрикой является скорость предсказания модели. Авторы отмечают, что размер KV-cache напрямую связан с итоговой скоростью работы, поэтому в своих экспериментах они фиксируют размер KV-cache и перебирают параметры по сетке.

После применения процедуры PostNAS авторы получают оптимальную конфигурацию гиперпараметров, отвечающих за внимание. Далее с учетом этой конфигурации происходит обучение, состоящее из двух шагов: дистилляция и полное дообучение. На этапе дистилляции модель с линейными слоями учится повторять распределение вероятностей исходной модели с помощью Дивергенции Кульбака—Лейблера. При

этом все параметры модели кроме тех, что используются в линейном внимании остаются замороженными. Этот этап отличается от предложенного в LoLCATs тем, что дистилляции происходит не послойно. На втором этапе происходит полное дообучение всей модели. В качестве данных для дистилляции авторы используют датасеты Nemotron-CC [20] и Redstone-QA [22], которые содержат в себе разнообразные данные, например: код, интернет-страницы, научные публикации и т.д. На втором этапе используется закрытый высококачественный датасет, состоящий из математических задач и кода.

2.2.3. Выводы

Классическое квадратичное внимание обладает вычислительной сложностью $\mathcal{O}(L^2)$ и требует линейно растущих объемов памяти для KV-кэша, что делает обработку длинных контекстов крайне ресурсоемкой. С другой стороны, классическое линейное внимание решает проблему сложности, но из-за ограниченной константной памяти теряет способность точно извлекать конкретные токены из длинной истории.

Среди различных модификаций линейного внимания наилучшие результаты демонстрирует архитектура Gated DeltaNet. Благодаря объединению механизма затухания и дельта-правила, она позволяет модели выборочно обновлять память и точнее извлекать нужную информацию из контекста.

Метод полной замены всех квадратичных слоев на линейные (LoLCATS) приводит к существенной деградации качества модели. Наблюдается значительное падение метрик на фактологических бенчмарках (например, MMLU) и полная неспособность модели справиться с задачей точного поиска в тексте (Passkey).

Метод Jet-Nemotron доказывает, что оптимальным решением является гибридная архитектура, в которой линейные слои чередуются с квадратичными. Такой подход позволяет значительно ускорить модель без потери качества, однако он требует внедрения специализированных алгоритмов (например, лучевого поиска) для выявления и сохранения наиболее важных слоев, отвечающих за конкретные навыки модели.

Для данной работы было принято решение использовать алгоритм обучения подобный LoLCATs, включающий фазы для послойной дистилляции и дообучения, поскольку в данном сценарии именно послойная дистилляция является наиболее эффективной задачей для обучения новых слоёв. А также включить фазу выбора важных слоёв и использовать гибридную архитектуру, как в Jet-Nemotron, поскольку линейные слои плохо справляются с некоторыми задачами, из-за чего требуется оставлять важные слои с квадратичным вниманием в модели.

3. Реализация

3.1. Описание метода

Метод, представленный в данной работе, основан на анализе публикаций и промежуточных результатах экспериментов. Он включает три этапа и сочетает преимущества подходов LoLCATs и Jet-Nemotron, но с рядом важных отличий. В предложенном подходе используется послойная дистилляция и дообучение, как в LoLCATs, и выбор важных слоёв, как в Jet-Nemotron. Важным отличием является то, что этап выбора слоёв производится над моделью, которую обучали без переключения между разными видами внимания. Это позволяет не тратить вычисления на построение дополнительной модели, как это делается в Jet-Nemotron. Также, в данной работе предложен иной алгоритм для выбора слоёв и для оценки важности используется две метрики, а не три. В качестве механизма линейного внимания, в отличие от LoLCATs, выбран Gated DeltaNet, поскольку он показывает себя лучше на бенчмарках, чем ядерное линейное внимание. Кроме того, в отличие от обеих упомянутых работ, для квадратичного внимания применяется Multi-Head Latent Attention вместо стандартного Multi-Head Attention. Метод состоит из следующих шагов:

1. **Послойная дистилляция.** На данном этапе к исходной модели добавляются слои с линейным вниманием параллельно квадратичному вниманию. Линейное внимание учится с помощью MSE на выходах исходного и нового слоёв. Также в качестве входов для следующих слоёв используется выход именно исходного внимания. В процессе дистилляции другие параметры модели не изменяются. При обучении на данном этапе используются данные, на которых обучалась исходная модель.
2. **Выбор конфигурации для гибридной модели.** После послойной дистилляции модель содержит в каждом слое исходное внимание и линейное. Конфигурация слоёв задаёт, какие из них

для этапа дообучения будут заменены на линейные. С помощью одного из двух предложенных алгоритмов подбирается наиболее удачная конфигурация для расположения линейных слоёв. Получаемая конфигурация должна приводить к наименьшей деградации метрик. До подбора конфигурации задаётся вычислительный бюджет — число исходных слоёв, которые не будут линейными, бюджет определяет необходимое число вычислений и объём памяти для конечной модели.

3. Дообучение. После дистилляции и выбора слоёв происходит дообучение с использованием кросс-энтропии в качестве функции потерь. Дообучение необходимо, чтобы модель научилась работать с новыми для неё слоями. В экспериментах использовалось несколько вариантов дообучения: обучение на исходном датасете и обучение на высококачественном датасете.

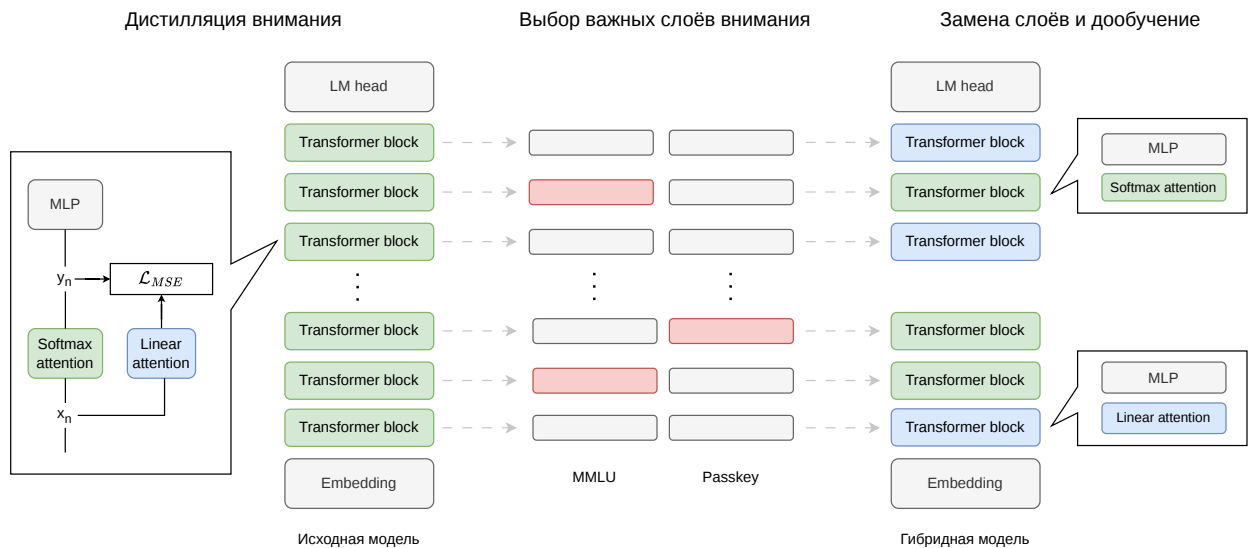


Рис. 4: Схема метода из трёх шагов.

Схема предложенного метода представлена на рис. 4. Слева изображена трансформерная архитектура с замененным слоем внимания, где параллельно исходному слою внимания добавляется новый с линейным вниманием, а также показано применение MSE. В середине изображена

модель с выбранными важными слоями. Справа конечная архитектура модели после выбора конфигурации расстановки слоёв.

3.2. Инструмент для выбора слоёв

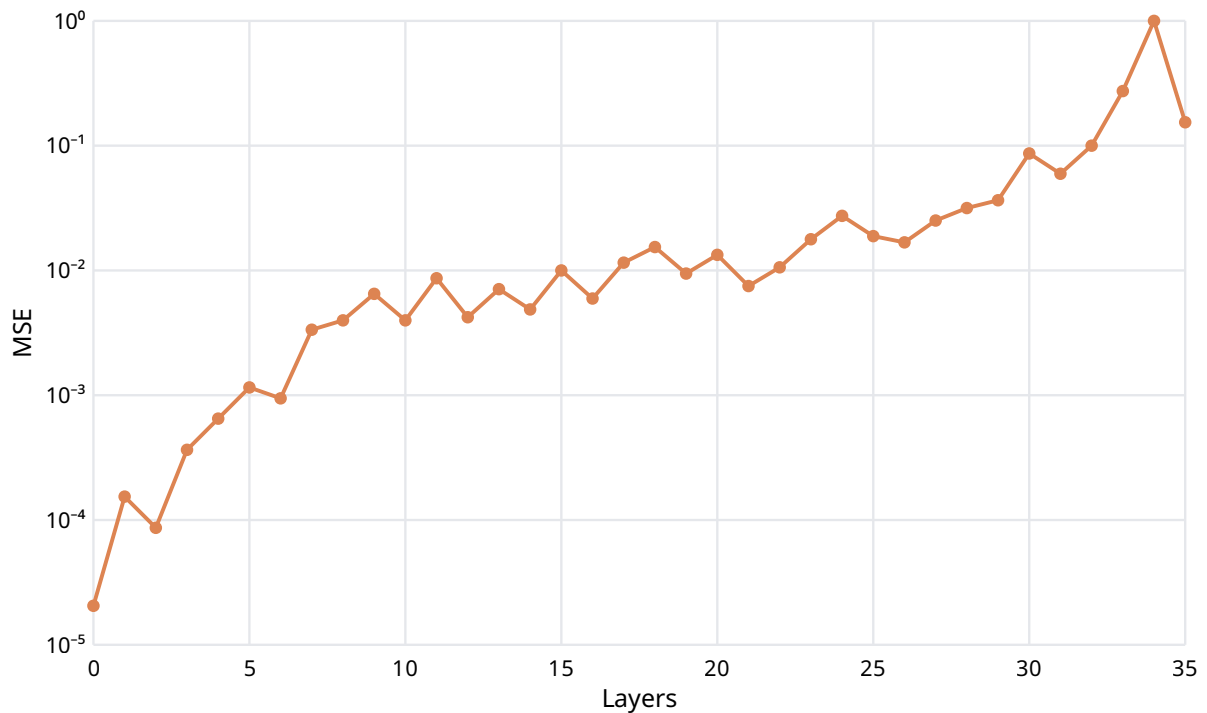


Рис. 5: MSE после дистилляции относительно слоя модели.

В процессе экспериментов было обнаружено, что замена некоторых квадратичных слоёв внимания на линейные приводит к значительному падению метрик, а дальнейшее дообучение не приводит к их восстановлению. Для преодоления этой проблемы можно использовать гибридную архитектуру, где часть слоёв внимания остаются исходными. При обучении гибридных моделей с нуля используется чередование, где каждые n слоёв остается исходное внимание. Этот же подход при дистилляции приводит к значительному падению метрик.

Для модели после дистилляции для каждого слоя была посчитана средняя ошибка на валидационной выборке из датасета. На рис. 5 представлен график зависимости MSE от номера слоя в модели. Видно, что для некоторых слоёв MSE может быть выше, что показывает, что

качество дистилляции зависит от слоя.

Также экспериментально было обнаружено, что существуют отдельные важные слои, которые отвечают за отдельные навыки модели: способность отвечать на фактологических бенчмарках, умение извлекать информацию из контекста и другие. Поэтому был создан отдельный инструмент для выбора таких слоёв. Для этого было реализовано два алгоритма: рандомизированный поиск и лучевой поиск. Результатом работы этих алгоритмов становится конфигурация модели, показывающая какие слои следует оставить с исходным вниманием, а какие заменить на линейное. Для работы этого инструмента необходима модель, для которой был выполнен первый этап дистилляции. Следующий этап дообучения происходит для модели, использующей эту конфигурацию.

3.2.1. Рандомизированный поиск

Рандомизированный алгоритм устроен следующим образом:

1. Фиксируется n случайных конфигураций с k слоями с квадратичным вниманием.
2. Для получившихся конфигураций замеряется одна из метрик.
3. Для каждого слоя подсчитывается среднее значение метрики в запусках, где он использовал квадратичное внимание.
4. Для каждой метрики выделяются m самых важных слоёв.
5. Полученный набор дальше сокращается с помощью перебора с отсечением менее важных слоёв.

3.2.2. Лучевой поиск

В алгоритме лучевого поиска заранее задаются два параметра: число слоёв с квадратичным вниманием n и количество поддерживаемых лучей k ; через l обозначим общее количество слоёв модели. На каждом шаге поддерживается k промежуточных конфигураций. Например, при $k = 2$ после трёх шагов это могут быть множества $\{1, 2, 3\}$ или $\{4, 5, 6\}$.

На очередной итерации каждая текущая конфигурация расширяется добавлением одного нового слоя. Таким образом, на шаге i формируется до $(l - i + 1) \times k$ кандидатов (фактически меньше, поскольку допускается добавление только слоёв с индексами, превышающими уже выбранные). Для каждого кандидата вычисляется целевая метрика, после чего сохраняются лишь k лучших вариантов.

Процесс повторяется до тех пор, пока не будут получены конфигурации, содержащие n слоёв. В результате можно либо выбрать конфигурацию с максимальным значением метрики, либо дополнительно уменьшить число слоёв с помощью полного перебора.

3.2.3. Метрики

Эксперименты, как в этой работе, так и в LoLCATs [12] показывают, что высокие показатели на фактологических бенчмарках не переносятся на бенчмарки, где необходимо извлекать информацию из контекста, особенно это заметно на более длинных последовательностях. Поэтому при использовании этих алгоритмов необходимо делать два запуска, один для выявления слоёв, отвечающих за фактологию и второй для слоёв, отвечающих за работу с контекстом. В данном случае используются валидационный набор MMLU [16] и Passkey [19].

MMLU — это набор тестовых задач в формате теста с множественным выбором из 4 вариантов. Он призван оценить эрудицию модели, насколько велики её знания различных фактов. На рис. 6 приведён пример задачи.

Passkey — это набор синтетических примеров, где внутри текста случайным образом помещается ключ, который модели необходимо обнаружить и выписать. Passkey проверяет способность модели извлекать необходимую информацию из контекста способность поддерживать эффективное внимание с увеличением длины контекста. На рис. 7 приведён пример задачи.

Одна из причин, по которой правительство препятствует возникновению монополий и регулирует их, заключается в том, что:

- (А) излишек производителя теряется, а излишек потребителя увеличивается. ✗
- (В) монопольные цены обеспечивают производственную эффективность, но лишают общество аллокативной эффективности. ✗
- (С) монопольные фирмы не занимаются значимыми исследованиями и разработками. ✗
- (D) излишек потребителя теряется из-за более высоких цен и низкого уровня выпуска продукции. ✓

Рис. 6: Пример задачи по микроэкономике из MMLU.

Тьма, пришедшая со Средиземного моря, накрыла ненавидимый прокуратором город. Исчезли висячие мосты, соединяющие храм со страшной Антониевой башней, **запомни ключ: 13452** опустилась с неба бездна и залила крылатых богов над гипсодромом, Хасмонеийский дворец с бойницами, базары, караван-сарай, переулки, пруды... **ключ: 13452**

Рис. 7: Пример Passkey задачи.

3.3. Модификация инфраструктуры

Для дистилляции внимания потребовалось модифицировать существующий фреймворк, использующийся при обучении больших языковых моделей. Для этого были реализованы дополнительные классы на PyTorch для слоёв внимания, которые позволяют дистиллировать внимание из одного вида в другой. Для Gated DeltaNet была использована библиотека Flash Linear Attention [32], реализующая эффективные ядра для различных видов линейного внимания на языке Triton [27]. Также была добавлена поддержка MSE — функции потерь для дистилляции, она применяется к выходам слоёв внимания, расположенным параллельно.

В листинге 1 представлен алгоритм прямого прохода внимания в процессе дистилляции. В процессе обучения градиенты вычисляются только для слоя линейного внимания, для того, чтобы не накапливать ошибку по мере прохождения сигнала вверх через модель, в качестве выхода используется выход исходного внимания. В процессе обучения каждые n шагов происходит оценка качества модели на валидационной выборке. Во время валидации используются выходы линейного внимания. В листинге 2 приведён алгоритм вычисления функции потерь, использующейся для дистилляции. Для каждого слоя вычисляется MSE над выходами слоёв внимания, затем полученные значения аккумулируются, усредняются по числу слоёв и домножаются на коэффициент масштабирования λ .

Листинг 1 Прямой проход слоя для дистилляции внимания

Input: Скрытые состояния H , маска внимания M

Output: Выход внимания, опциональная пара из выходов двух вариантов внимания

```

1: if training mode then
2:   Disable gradient computation
3:    $A \leftarrow \text{BASEATTENTION}(H, M)$      $\triangleright$   $A$  выходы квадратичного внимания
4:   Enable gradient computation
5:    $G \leftarrow \text{GATEDDELTA NET}(H, M)$      $\triangleright$   $G$  выходы линейного внимания
6:   return  $A, (A, G)$ 
7: else
8:    $G \leftarrow \text{GATEDDELTA NET}(H, M)$ 
9:   return  $G, \text{None}$ 
10: end if
```

Листинг 2 Вычисление функции потерь для дистилляции внимания

Input: O — множество выходов слоёв внимания $\{(A_l, G_l)\}_{l=1}^L$, где A_l — выход исходного внимания, G_l — выход Gated DeltaNet, L — количество слоёв; λ — коэффициент масштабирования).

Output: Итоговое значение функции потерь $\mathcal{L}_{\text{distill}}$.

```
1:  $\mathcal{L}_{\text{sum}} \leftarrow 0$ 
2: for  $l = 1$  to  $L$  do
3:    $(A_l, G_l) \leftarrow O_l$ 
4:    $\mathcal{L}_{\text{MSE}}^{(l)} \leftarrow \frac{1}{N} \sum_{i=1}^N (G_{l,i} - A_{l,i})^2$   $\triangleright$  MSE для текущего слоя
5:    $\mathcal{L}_{\text{sum}} \leftarrow \mathcal{L}_{\text{sum}} + \mathcal{L}_{\text{MSE}}^{(l)}$ 
6: end for
7:  $\mathcal{L}_{\text{distill}} \leftarrow \lambda \cdot \frac{\mathcal{L}_{\text{sum}}}{L}$   $\triangleright$  Усреднение и масштабирование
8: return  $\mathcal{L}_{\text{distill}}$ 
```

Помимо этого, для возможности использовать модель после обучения возникла необходимость в модификации фреймворка для инференса — SGLang [25], чтобы поддержать новую архитектуру, поскольку в нём не были поддержаны гибридные модели, использующие Multi-Head Latent Attention. Полученная реализация переиспользует классы SGLang для моделей DeepSeek-V2 и Qwen3-Next. Также была добавлена поддержка Multi-Token Prediction (МТР) блока [2] [6], использующего линейное внимание. МТР позволяет модели предсказывать сразу несколько токенов за один прямой проход, что позволило ускорить генерацию токенов на 33%.

Для обучения на последовательностях больше 4096 потребовалось реализовать алгоритм Sequence Parallelism [7] для линейного внимания, поскольку на больших контекстах активации перестают помещаться в одну GPU. Алгоритм устроен следующим образом:

- До обучения для каждого экземпляра модели создаётся группа из n процессов.
- Входящая последовательность разделяется на n частей, каждая часть передается одному из процессов.
- Каждый ранг обрабатывает свою часть отдельно кроме слоёв внимания. В слоях внимания входящий тензор из внутренних

представлений разделяется по числу голов внимания. Далее с помощью операции all-to-all каждый ранг получает активации k голов с полной длиной последовательности. Далее на каждом ранге подсчитывается внимание для k голов. Затем с помощью ещё одного all-to-all каждый ранг получает активации для всех голов, но только для части последовательности.

3.4. Результат

В результате были реализованы следующие компоненты.

- **Разработан трехэтапный метод замены внимания** послойная дистилляция, выбор конфигурации, дообучение.
- **Создан инструмент для выбора важных слоев** на основе рандомизированного и лучевого поиска. Это позволяет выявлять и сохранять слои, критически важные для фактологических знаний и извлечения информации из контекста.
- **Модифицирована инфраструктура** в обучающий пайплайн на PyTorch интегрированы ядра на Triton (Flash Linear Attention), а фреймворк инференса SGLang адаптирован для работы с гибридной архитектурой и Multi-Token Prediction, использующим линейное внимание.
- **Реализован алгоритм Sequence Parallelism** для линейного внимания, что позволило преодолеть ограничения по памяти и масштабировать обучение на последовательности длиной свыше 4096 токенов.

Полученная реализация позволила эффективно преобразовывать модели, заменяя исходную на гибридную используя три этапа, а также эффективно оценивать модель и использовать её для инференса.

4. Эксперименты

4.1. Исследовательские вопросы

Целью данного экспериментального исследования является поиск ответов на следующие исследовательские вопросы.

- **RQ1:** *Какого качества можно добиться с помощью предложенного метода и нужны ли все описанные этапы для наилучшего результата?*
 - **RQ1.1:** *Есть ли необходимость в гибридной архитектуре?*
 - **RQ1.2:** *Сколько слоёв необходимо оставить с квадратичным вниманием?*
 - **RQ1.3:** *Нужен ли этап дообучения?*
 - **RQ1.4:** *Влияет ли выбор датасета на качество при дообучении?*
 - **RQ1.5:** *Возможно ли восстановить исходное качество модели?*
- **RQ2:** *Как изменяется производительность модели во время обучения и инференса?*
 - **RQ2.1:** *Насколько отличается производительность при обучении?*
 - **RQ2.2:** *Насколько полученная модель быстрее на инференсе?*

4.2. Дизайн экспериментов

Для проведения экспериментов использовалась закрытая предобученная модель с 3 млрд. параметров. Исходная модель имеет архитектуру схожую с Llama 3 [33] за исключением использования Multi-Head Latent Attention [6] в качестве слоёв внимания. Всего в модели 36 слоёв. Гиперпараметры для линейного внимания подбираются так, чтобы число

параметров в слоях внимания не изменялось значительно, исключая эффекты от увеличения или уменьшения общего числа параметров.

В качестве данных для дистилляции был выбран исходный датасет, который использовался в процессе предобучения модели. Этот датасет включает в себя разнообразные данные: интернет-страницы, код, научные публикации и т.д. Для дообучения использовались как данные из исходного датасета, так и концентрированный высококачественный датасет Nemotron-CC-Math-v1 [21]. Этот датасет содержит в себе качественные примеры с математическими задачами и кодом. Дистилляция выполнялась на 20 млрд. токенов, а дообучение на 50 млрд, что составляет всего 0,46% от исходного датасета. Для валидации в процессе дистилляции используется небольшое подмножество исходного датасета размером 40 млн., на нём подсчитывается валидационная кросс-энтропия. Контекст обучения — 4096 токенов.

Все эксперименты проводились с использованием модифицированной версии закрытого фреймворка для обучения с поддержкой FSDP на 32 NVIDIA H100, объединенных с помощью NVLink. Обучение происходит без использования varlen во внимании, то есть при обучении не вводятся дополнительных масок для внимания над конкатенированными примерами из-за особенностей реализации.

4.3. Описание используемых бенчмарков

Для оценки качества получаемых моделей использовался следующий набор бенчмарков:

- **MMLU [16]**. Оценка базовых знаний модели в широком спектре академических дисциплин: от гуманитарных наук до юриспруденции и медицины.
- **ruMMLU [15]**. Русскоязычная адаптация оригинального теста MMLU. Оценивает не только общие академические знания, но и то, насколько хорошо модель справляется с контекстом, спецификой и языковыми особенностями русского языка.

- **MMLU-Pro [18]**. Усложненная и расширенная модификация стандартного MMLU. Включает задачи, требующие более глубокого логического рассуждения и экспертных знаний, при этом количество ложных вариантов ответа существенно увеличено.
- **HellaSwag [9]**. Бенчмарк для проверки понимания физического мира и здравого смысла. Модели предлагается продолжить описание повседневной ситуации, выбрав наиболее логичный и естественный для человека финал истории.
- **BoolQ [3]**. Тест на понимание прочитанного текста. Модель анализирует небольшой текстовый фрагмент и отвечает на естественные вопросы в формате «да/нет».
- **Winogrande [30]**. Исследование способности модели разрешать двусмысленности в тексте. Задача требует понимания скрытого контекста и логики взаимодействия предметов или людей для определения того, к какому именно слову относится местоимение.
- **LIBRA [13]**. Бенчмарк предназначенный для проверки способностей языковых моделей работать на длинных задачах на русском языке.

В названиях бенчмарков в таблицах также присутствуют отметки о числе дополнительных примеров в задачах при оценке. 0-shot означает, что модели на вход подавалась только задача без примеров. 5-shot означает, что перед задачей также подавалось 5 примеров.

4.4. Обучение модели

4.4.1. Дистилляция

На данном этапе к исходной модели в каждом слое внимания параллельно добавляется модуль с Gated DeltaNet. Параметры Gated DeltaNet инициализируются случайным образом с учётом особенностей данной архитектуры, чтобы обеспечить стабильность при обучении. При обучении на данном этапе линейное внимание в каждом слое учится

аппроксимировать выходы исходного внимания. Обучение происходит на исходном датасете.

На рис. 8 представлен график для кросс-энтропии на валидационной выборке во время дистилляции. Во время замера валидации исходные слои внимания выключаются и сигнал проходит только через слои с линейным вниманием. Это позволяет оценить, насколько хорошо модель восстанавливает своё качество на задаче языкового моделирования в процессе дистилляции. На графике можно заметить, что в начале дистилляции функция потерь принимает достаточно большие значения и затем плавно снижается до значений соответствующим предобученной модели.

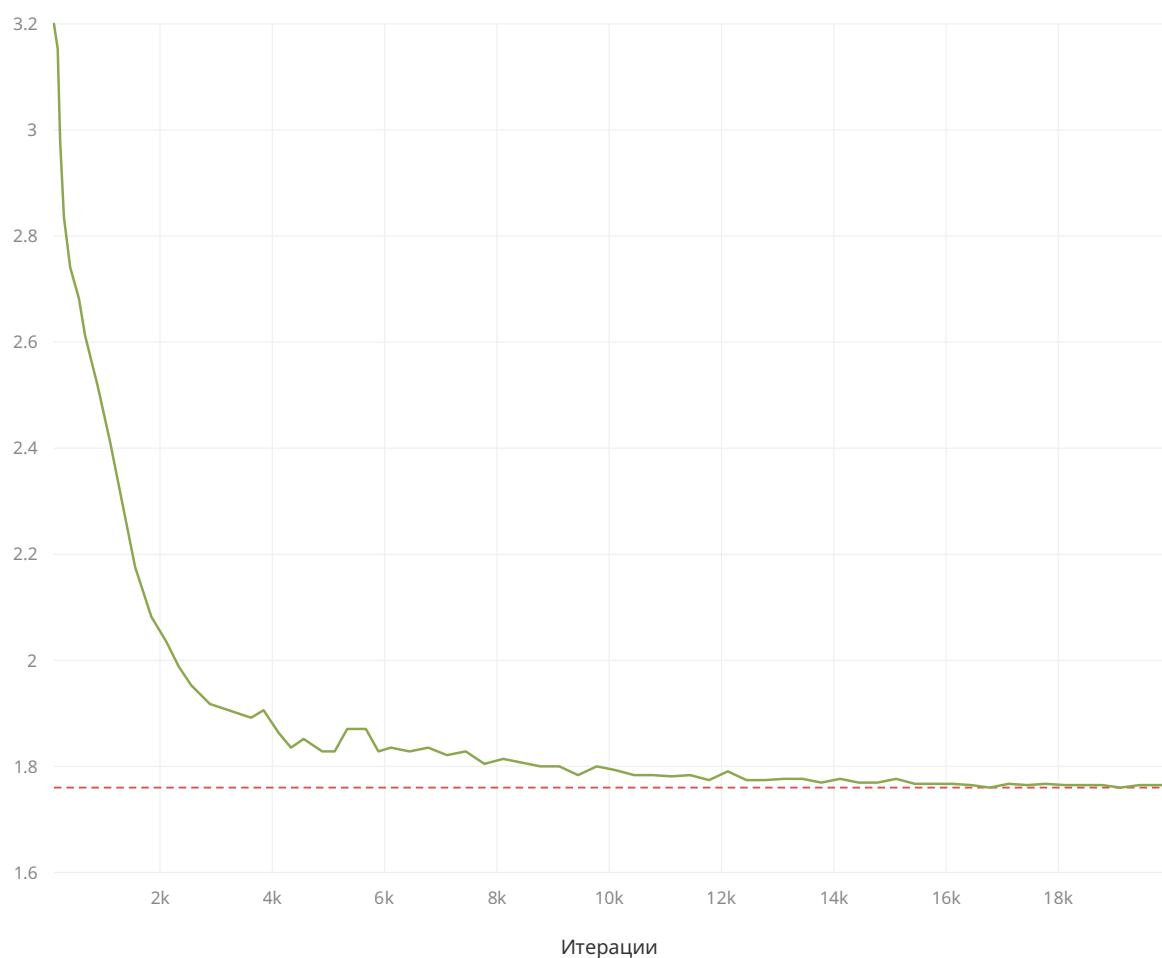


Рис. 8: Зелёная линия — кросс-энтропия на валидационной выборке во время дистилляции, красная линия — кросс-энтропия предобученной модели в конце обучения.

Таблица 2: Сравнение метрик на основных бенчмарках для исходной модели и модели после дистилляции со всеми линейными слоями.

Метрика	Исходная модель	После дистилляции	Изменение
MMLU 0-shot	0,609	0,436	−28,41%
MMLU 5-shot	0,625	0,376	−39,84%
ruMMLU 5-shot	0,573	0,339	−40,84%
MMLU-Pro 0-shot	0,291	0,139	−52,23%
MMLU-Pro 5-shot	0,306	0,130	−57,52%
HellaSwag 0-shot	0,742	0,713	−3,91%
BoolQ 0-shot	0,819	0,663	−19,05%
Winogrande 0-shot	0,716	0,669	−6,56%

В таблице 2 представлены значения метрик для модели после дистилляции, в которой все слои внимания являются линейными. Можно заметить, что все метрики значительно упали, особенно для MMLU-Pro, значение которого приблизилось к уровню случайного угадывания: 0,1. Поэтому возникает необходимость использования гибридной архитектуры.

4.4.2. Выбор важных слоёв

На этапе выбора важных слоёв использовался рандомизированный алгоритм. Наиболее хороший результат при поиске был получен с использованием 7 слоёв квадратичного внимания, 4 из них отвечают за общие знания: {15, 18, 23, 25}, а 3 за работу с контекстом: {3, 7, 11}. В этой конфигурации модель получила наиболее высокие метрики на валидационных бенчмарках MMLU и Passkey.

На рис. 9 представлена карта с оценками важности слоёв модели на MMLU, а на рис. 10 на Passkey. Можно заметить, что важные слои распределены в модели по-разному. Слои, важные для работы с контекстом, расположены ближе к началу модели. А слои, отвечающие за различные знания, находятся в середине. Также видно, что стратегия наивного чередования может не покрыть важные слои и приводить к

худшим результатам.

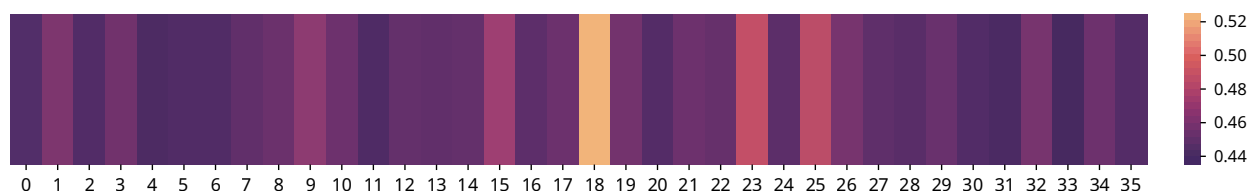


Рис. 9: Карта важности слоёв для MMLU. Яркость соответствует важности слоя для метрики.

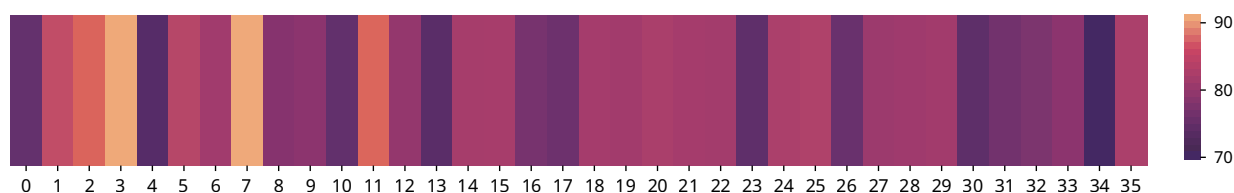


Рис. 10: Карта важности слоёв для Passkey. Яркость соответствует важности слоя для метрики.

В таблице 3 представлены метрики для гибридной модели до этапа дообучения. Гибридный вариант значительно превосходит модель, в которой используется только линейное внимание, но всё ещё уступает исходной модели.

Таблица 3: Сравнение метрик на основных бенчмарках для исходной модели и гибридной модели с 7 квадратичными слоями до дообучения.

Метрика	Исходная модель	Без дообучения	Изменение
MMLU 0-shot	0,609	0,573	−5,91%
MMLU 5-shot	0,625	0,581	−7,04%
ruMMLU 5-shot	0,573	0,523	−8,73%
MMLU-Pro 0-shot	0,291	0,247	−15,12%
MMLU-Pro 5-shot	0,306	0,256	−16,34%
HellaSwag 0-shot	0,742	0,723	−2,56%
BoolQ 0-shot	0,819	0,763	−6,84%
Winogrande 0-shot	0,716	0,698	−2,51%

4.4.3. Дообучение

Для того, чтобы дополнительно проверить правильность метода выбора важных слоёв было выполнено дообучение с разным числом слоёв квадратичного внимания. Результаты приведены в таблице 4. Увеличение числа слоёв не приводит к значительному росту метрик, и наоборот уменьшение числа слоёв в экспериментах приводило к падению метрик. Для этих экспериментов использовалось меньшее число токенов — 10 млрд. и исходный датасет.

Таблица 4: Сравнение метрик на основных бенчмарках с изменением числа слоёв с квадратичным вниманием.

Метрика	4 слоя	7 слоёв	10 слоёв	18 слоёв
MMLU 0-shot	0,5627	0,5821	0,5865	0,5898
MMLU 5-shot	0,5710	0,5958	0,5936	0,6005
ruMMLU 5-shot	0,5187	0,5388	0,5432	0,5467
MMLU-Pro 0-shot	0,2384	0,2524	0,2541	0,2547
MMLU-Pro 5-shot	0,2403	0,2719	0,2743	0,2755
HellaSwag 0-shot	0,7263	0,7295	0,7300	0,7340
BoolQ 0-shot	0,7669	0,7896	0,7865	0,7899
Winogrande 0-shot	0,7103	0,7151	0,7143	0,7056

Итоговое дообучение было выполнено с использованием исходного датасета, а также смешанного с добавлением концентрированного качественного датасета. В таблице 5 приведены метрики для исходной модели, двух вариантов после дообучения и модели до дообучения. Можно заметить, что дообучение позволяет дополнительно приблизиться к значениям исходной модели. Также можно заметить, что смешанный датасет позволяет дополнительно улучшить показатели для MMLU-Pro.

Таблица 5: Сравнение метрик на основных бенчмарках для исходной модели, дообученной на смешанном датасете, дообученной на исходном датасете и для модели без дообучения.

Метрика	Исходная модель	Смешанный датасет	Исходный датасет	Без дообучения
MMLU 0-shot	0,609	0,595	0,587	0,573
MMLU 5-shot	0,625	0,605	0,601	0,581
ruMMLU 5-shot	0,573	0,554	0,546	0,523
MMLU-Pro 0-shot	0,291	0,271	0,255	0,247
MMLU-Pro 5-shot	0,306	0,294	0,277	0,256
HellaSwag 0-shot	0,742	0,732	0,733	0,723
BoolQ 0-shot	0,819	0,804	0,801	0,763
Winogrande 0-shot	0,716	0,706	0,715	0,698

4.5. Результаты обучения

В таблице 6 приведены финальные метрики на основных бенчмарках для модели после прохождения всех этапов обучения. Полученная модель показывает сопоставимые метрики с исходной моделью, что говорит об успешном восстановлении способностей после замены внимания.

Таблица 6: Сравнение метрик на основных бенчмарках для исходной модели и полученной из неё гибридной модели.

Метрика	Исходная модель	Гибридная модель	Изменение
MMLU 0-shot	0,609	0,595	−2,30%
MMLU 5-shot	0,625	0,605	−3,20%
ruMMLU 5-shot	0,573	0,554	−3,32%
MMLU-Pro 0-shot	0,291	0,271	−6,87%
MMLU-Pro 5-shot	0,306	0,294	−3,92%
HellaSwag 0-shot	0,742	0,732	−1,35%
BoolQ 0-shot	0,819	0,804	−1,83%
Winogrande 0-shot	0,716	0,706	−1,40%

В таблице 7 представлены результаты на задаче Passkey из бенчмарка LIBRA на контекстах с длиной от 4096 до 65536. Гибридная модель смогла превзойти исходную на контексте длиной 65536.

Таблица 7: Сравнение метрики Passkey из LIBRA для исходной модели и полученной из неё гибридной модели.

Модель	4k	8k	16k	32k	64k
Исходная модель	1,000	1,000	1,000	1,000	0,705
Гибридная модель	1,000	1,000	1,000	1,000	1,000

При замере всех задач на Libra, гибридная модель получила значение 0,258 против 0,275 исходной модели.

4.6. Оценка производительности

Для того, чтобы убедиться, что во время обучения Gated DeltaNet не требует чрезмерного потребления ресурсов, были сделаны замеры для прямого и обратного проходов. Замеры выполнялись для двух наборов гиперпараметров внимания, соответствующих моделям с 3 млрд. и 10 млрд параметров. Между ними отличаются следующие гиперпараметры: размер скрытого состояния, число голов внимания, размеры скрытых представлений голов.

Видно, что в случае для модели с 3 млрд. параметров происходит замедление до 24%, это связано с присутствием дополнительных расходов на вызов Triton ядер. Из-за небольших размеров тензоров вычисления на GPU происходят быстрее, чем запуск ядер на CPU. Этой проблемы нет для моделей большего размера. В случае для модели с 10 млрд. параметров замедление составляет 1%, что является приемлемым для предложенного метода. В предложенном сравнении в Multi-Head Latent Attention используется FlashAttention-2 [4].

Таблица 8: Сравнение производительности в конфигурации для модели с 3 млрд. параметров.

Внимание	forward avg (ms)	forward std (ms)	backward avg (ms)	backward std (ms)	max reserved memory (GB)
MLA	1,228	0,016	1,822	0,018	0,37
GDN	1,434	0,015	2,262	0,010	0,78

Таблица 9: Сравнение производительности в конфигурации для модели с 10 млрд. параметров.

Внимание	forward avg (ms)	forward std (ms)	backward avg (ms)	backward std (ms)	max reserved memory (GB)
MLA	1,545	0,015	2,706	0,030	0,75
GDN	1,565	0,004	2,859	0,013	1,11

Также для полученной реализации архитектуры в SGLang была оценена скорость генерации токенов в двух сценариях: типовой пользовательский запрос через чат-бот и работа с длинным контекстом. В первом сценарии необходимо для последовательности 8192 токенов сгенерировать 1024 токена, во втором для контекста 131072 сгенерировать 1024 токена. Для оценки использовалась модель с параметрами соответствующими конфигурации финальной обученной модели, то есть 3 млрд. параметров, 36 слоёв и 7 из них оставлены исходными. Для первого сценария ускорение составило 19,82%, для второго 37,32%, разница в ускорении связана с квадратичным вниманием, для которого характерно замедление работы по мере увеличения контекста.

4.7. Выводы

Таким образом, на поставленные исследовательские вопросы можно ответить следующим образом.

- **RQ1:** *Какого качества можно добиться с помощью предложенного метода и нужны ли все описанные этапы для наилучшего результата?*

- **RQ1.1:** *Есть ли необходимость в гибридной архитектуре?*

Да, поскольку модель без квадратичного внимания теряет свои способности и показывает плохие результаты на бенчмарках.

- **RQ1.2:** *Сколько слоёв необходимо оставить с квадратичным вниманием?*

Для данной модели оптимальным числом является 7 слоёв, что составляет приблизительно пятую часть от всех слоёв модели, что в свою очередь означает уменьшение потребления памяти в пять раз. Увеличение числа слоёв не улучшает качество, при этом увеличивая потребление памяти. Дальнейшее уменьшение приводит к ухудшению качества.

- **RQ1.3:** *Нужен ли этап дообучения?*

Да, он позволяет дополнительно восстановить исходные метрики модели.

- **RQ1.4:** *Влияет ли выбор датасета на качество при дообучении?*

Да, более качественные данные позволяют дополнительно приблизиться к качеству исходной модели.

- **RQ1.5:** *Возможно ли восстановить исходное качество модели?*

На различных бенчмарках гибридная модель показывает сопоставимые метрики с исходной моделью, практически полностью восстанавливая изначальное качество.

- **RQ2:** *Как изменяется производительность модели во время обучения и инференса?*

- **RQ2.1:** *Насколько отличается производительность при обучении?*

Из-за особенностей Triton обучение для модели с 3 млрд. параметров замедляется на 24%. С увеличением числа параметров

данное замедление исчезает, что говорит о возможности масштабирования метода на модели большего размера.

– **RQ2.2:** *Насколько полученная модель быстрее на инференсе?*

Скорость предсказаний выросла на 19,82% для типовых сценариев использования. При работе с длинными последовательностями ускорение составило 37,32%.

Предложенный метод преобразования квадратичного внимания в линейное продемонстрировал высокую эффективность. Гибридная модель показывает метрики, сопоставимые с исходной моделью на классических бенчмарках. На длинных контекстах в задаче Passkey (64 тыс. токенов) гибридная архитектура превзошла исходную. Модель сохранила абсолютную точность извлечения, показав результат 1,000 против 0,705 у исходной архитектуры. Новая архитектура позволила сократить потребление памяти в 5 раз. Скорость предсказаний выросла на 19,82% для типовых сценариев использования. При работе с длинными последовательностями ускорение составило 37,32%. Вычислительные затраты на преобразование составили лишь 0,46% от ресурсов, потраченных на предобучение исходной модели.

Заключение

В ходе работы были достигнуты следующие результаты.

1. Выполнен обзор существующих решений для замены квадратичного внимания на линейное в предобученных больших языковых моделях.
2. Предложен метод для замены внимания. Реализованы все необходимые инфраструктурные компоненты: добавлена поддержка дистилляции в фреймворк обучения, реализован алгоритм для обучения на длинных последовательностях, новая архитектура добавлена в фреймворк для инференса, а также реализован инструмент для выбора важных слоёв.
3. Преобразована модель с помощью предложенного метода и проведено экспериментальное исследование. Оценка на широком наборе фактологических бенчмарков (включая MMLU, HellaSwag и BoolQ), а также на бенчмарке для оценки качества работы с длинными последовательностями Libra, показала, что полученная модель демонстрирует метрики, сопоставимые с оригинальной моделью. При этом для работы ей требуется в пять раз меньше памяти, также она быстрее на этапе предсказания на 19,82% в типовых сценариях использования и на 37,32% при работе с длинными последовательностями. Преобразование модели предложенным методом потребовало лишь 0,46% от вычислений, потраченных на предобучение исходной модели, что говорит об эффективности метода.

Список литературы

- [1] Attention is all you need / Ashish Vaswani, Noam Shazeer, Niki Parmar et al. // Advances in neural information processing systems. — 2017. — Vol. 30.
- [2] Better & faster large language models via multi-token prediction / Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière et al. // arXiv preprint arXiv:2404.19737. — 2024.
- [3] Boolq: Exploring the surprising difficulty of natural yes/no questions / Christopher Clark, Kenton Lee, Ming-Wei Chang et al. // Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers). — 2019. — P. 2924–2936.
- [4] Dao Tri. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning // International Conference on Learning Representations (ICLR). — 2024.
- [5] Dao Tri, Gu Albert. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality // arXiv preprint arXiv:2405.21060. — 2024.
- [6] Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model / Aixin Liu, Bei Feng, Bin Wang et al. // arXiv preprint arXiv:2405.04434. — 2024.
- [7] Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models / Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang et al. // arXiv preprint arXiv:2309.14509. — 2023.
- [8] Gated linear attention transformers with hardware-efficient training / Songlin Yang, Bailin Wang, Yikang Shen et al. // Proceedings of the 41st International Conference on Machine Learning. — ICML'24. — JMLR.org, 2024. — 23 p.

- [9] Hellaswag: Can a machine really finish your sentence? / Rowan Zellers, Ari Holtzman, Yonatan Bisk et al. // Proceedings of the 57th annual meeting of the association for computational linguistics. — 2019. — P. 4791–4800.
- [10] Jet-nemotron: Efficient language model with post neural architecture search / Yuxian Gu, Qinghao Hu, Shang Yang et al. // arXiv preprint arXiv:2508.15884. — 2025.
- [11] Kimi linear: An expressive, efficient attention architecture / Kimi Team, Yu Zhang, Zongyu Lin et al. // arXiv preprint arXiv:2510.26692. — 2025.
- [12] Lolcats: On low-rank linearizing of large language models / Michael Zhang, Simran Arora, Rahul Chalamala et al. // arXiv preprint arXiv:2410.10254. — 2024.
- [13] Churin Igor, Apishev Murat, Tikhonova Maria et al. Long Input Benchmark for Russian Analysis. — 2024. — [2408.02439](#).
- [14] Lora: Low-rank adaptation of large language models. / Edward J Hu, Yelong Shen, Phillip Wallis et al. // Iclr. — 2022. — Vol. 1, no. 2. — P. 3.
- [15] MERA: A comprehensive LLM evaluation in Russian / Alena Fenogenova, Artem Chervyakov, Nikita Martynov et al. // Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). — 2024. — P. 9920–9948.
- [16] Measuring Massive Multitask Language Understanding / Dan Hendrycks, Collin Burns, Steven Basart et al. // International Conference on Learning Representations. — 2021. — URL: <https://openreview.net/forum?id=d7KBjmI3GmQ>.
- [17] Jiang Albert Q., Sablayrolles Alexandre, Mensch Arthur et al. Mistral 7B. — 2023. — [2310.06825](#).

- [18] Mmlu-pro: A more robust and challenging multi-task language understanding benchmark / Yubo Wang, Xueguang Ma, Ge Zhang et al. // Advances in Neural Information Processing Systems. — 2024. — Vol. 37. — P. 95266–95290.
- [19] Mohtashami Amirkeivan, Jaggi Martin. Landmark Attention: Random-Access Infinite Context Length for Transformers. — 2023. — [2305.16300](#).
- [20] Nemotron-cc: Transforming common crawl into a refined long-horizon pretraining dataset / Dan Su, Kezhi Kong, Ying Lin et al. // Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). — 2025. — P. 2459–2475.
- [21] Rabeeh Karimi Mahabadi Sanjeev Satheesh Shrimai Prabhumoye Mostofa Patwary Mohammad Shoeybi Bryan Catanzaro. Nemotron-cc-math: A 133 Billion-Token-Scale High Quality Math Pretraining Dataset. — 2025. — URL: <https://arxiv.org/abs/2508.15096>.
- [22] Redstone: Curating general, code, math, and qa data for large language models / Yaoyao Chang, Lei Cui, Li Dong et al. // arXiv preprint arXiv:2412.03398. — 2024.
- [23] Schlag Imanol, Irie Kazuki, Schmidhuber Jürgen. Linear transformers are secretly fast weight programmers // International conference on machine learning / PMLR. — 2021. — P. 9355–9366.
- [24] Sequential-niah: A needle-in-a-haystack benchmark for extracting sequential needles from long contexts / Yifei Yu, Qian-Wen Zhang, Lingfeng Qiao et al. // Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. — 2025. — P. 29438–29456.
- [25] Sglang: Efficient execution of structured language model programs / Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie et al. // Advances in neural information processing systems. — 2024. — Vol. 37. — P. 62557–62583.

- [26] Team Qwen. Qwen3.5: Accelerating Productivity with Native Multi-modal Agents. — 2026. — February. — URL: <https://qwen.ai/blog?id=qwen3.5>.
- [27] Tillet Philippe, Kung Hsiang-Tsung, Cox David. Triton: an intermediate language and compiler for tiled neural network computations // Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages. — 2019. — P. 10–19.
- [28] Transformers are rnns: Fast autoregressive transformers with linear attention / Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, François Fleuret // International conference on machine learning / PMLR. — 2020. — P. 5156–5165.
- [29] Widrow Bernard, Hoff Marcian E. Adaptive switching circuits // Neurocomputing: foundations of research. — 1988. — P. 123–134.
- [30] Winogrande: An adversarial winograd schema challenge at scale / Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, Yejin Choi // Communications of the ACM. — 2021. — Vol. 64, no. 9. — P. 99–106.
- [31] Yang Songlin, Kautz Jan, Hatamizadeh Ali. Gated delta networks: Improving mamba2 with delta rule // arXiv preprint arXiv:2412.06464. — 2024.
- [32] Yang Songlin, Zhang Yu. FLA: A Triton-Based Library for Hardware-Efficient Implementations of Linear Attention Mechanism. — 2024. — . — URL: <https://github.com/fla-org/flash-linear-attention>.
- [33] The llama 3 herd of models / Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri et al. // arXiv preprint arXiv:2407.21783. — 2024.